

에이전트 제어 평면 업그레이드 계획 — 참조 설계 기반 (CP-5 ~ CP-8)

← [에이전트 제어 평면 설계](#) · [아키텍처](#) · [개발 계획서](#)

0. 현황 — v1(CP-1~CP-4) 구현 완료

`nabi-control` 크레인트(protocol/server/dispatch/policy/subscribe/client)로 다음이 **동작·실사용** 검증됨:

영역	구현 상태
전송	named pipe NDJSON(<code>\\.\pipe\nabi-control-<PID></code>), Hello 토큰 인증, env 주입 (<code>NABI_PANE_ID</code> / <code>NABI_CONTROL_PIPE</code> / <code>NABI_CONTROL_TOKEN</code>)
질의	<code>list</code> (id-제목·크기), <code>capture</code> (화면+스크롤백 N줄, 커서 위치)
동작	<code>spawn</code> (shell/cwd), <code>send</code> (raw), <code>kill</code> , <code>resize</code> , <code>open-browser</code> , <code>open-sftp</code> (저장 세션)
구독	<code>wait --until</code> exit/command-done/output/idle, <code>tail</code> (출력 스트림)
정책	off/ask/on + 요청자 pane별 1회 승인(토스트), tracing 감사 로그

v1은 "에이전트가 옆 pane에 명령 띄우고 결과 관찰" 시나리오를 성립시켰다. 본 문서는 참조 설계 (tmux·WezTerm·Kitty·iTerm2·Herdr·cmux) 분석으로 v1의 갭을 메우고 에이전트 1급 시민 터미널로 끌어올리는 계획이다.

1. 참조 설계 분석 — 무엇을 배우나

참조	핵심 기능	nabi에 가져올 것
tmux	<code>capture-pane -p -e -S/-E</code> (범위·이스케이프), <code>pipe-pane</code> (출력 파이프), <code>wait-for</code> , 타깃 문법	capture 범위 지정·SGR 보존 옵션, tail의 델타 스트림화,

참조	핵심 기능	nabi에 가져올 것
	(session:win.pane), hooks	pane 종료/벨 혹은 알림
WezTerm	cli spawn / split-pane --right/--bottom/--top-level (pane ID 회신), send-text (bracketed paste 존 중), list --format json, activate-pane	DockTarget (탭/분할/새 창) 스폰, --json 출력 모드, send의 paste 의미론, focus verb
Kitty	kitten @ --match "title:x cwd:y" (속성 매칭), remote_control_password +액션 단위 권한(--rc-permission), 창별 제어 제한	--match 주소지정, verb 그룹별 권한 정책, pane별 제어 허용/차단
iTerm2	Python API: 세션 변수 구독, 비동기 모니터(출력/프롬프트/변수), 트리거	변수형 pane 메타데이터 (cwd-exit code·명령)의 구독 가능한 단일 모델
Herdr	에이전트 상태(working/idle/blocked) 사이드바 상시 표시	pane 활동 상태 머신 을 list 응답·탭 UI 양쪽에 노출
cmux	Unix socket API, 에이전트 알림·진행 보고, 오케스트레이터 시가 다중 에이전트 스폰	notify verb (에이전트→사용자 토스트), MCP 서버 노출 (에이전트 네이티브 도구화)

공통 패턴: ① 머신 가독 출력(JSON)은 기본 소양, ② 주소지정은 ID+속성 매칭 병행, ③ 권한은 액션 단위가 업계 표준(Kitty), ④ 2026 에이전트 도구들은 "상태 인지(busy/idle/blocked)"를 1급으로 다룸.

2. 겹 분석 — v1 코드 리뷰 결과

2.1 정합성 결함 (즉시 수정 대상)

#	결함	위치	내용
G1	스폰 ID 폴링 레이스	dispatch.rs	새 pane ID를 before/after 집합 차이로 폴링 — 동시 스폰 2건이면 ID가 뒤바뀔 수 있음. Command::SpawnLocalPane 에 응답 oneshot(또는 seq 태그) 추가로 정확한 ID 회신
G2	tail이 델타가 아님	server.rs	출력 변경 시 마지막 40줄을 통째로 재덤프 — 중복 출력·빠른 스크롤 유실. VT damage/스크롤백 오프셋 기반 새 줄만 전송

#	결함	위치	내용
G3	wait가 데이터를 버림	<code>subscribe.rs</code>	<code>until exit</code> 의 종료 코드, <code>command-done</code> 의 <code>CommandBlock</code> (명령·exit code·소요시간)이 이벤트에 있는데 <code>data: ""</code> 로 응답. 이벤트 페이로드를 data(JSON)로 회신
G4	wait/tail 정책 분류 오류	<code>server.rs</code>	읽기 전용 구독인데 쓰기 정책(<code>allow_write</code>) 게이트 — <code>capture</code> 는 자유인데 <code>tail</code> 은 승인 필요한 비밀관. 읽기로 재분류 (§4 정책 재설계와 함께)
G5	파이프 ACL 미명시	<code>server.rs</code>	<code>ServerOptions::new().create()</code> 기본 보안 기술자 사용 — 현재 사용자 SID 전용 DACL 명시 필요(설계 문서 §4 약속 사항)
G6	send의 paste 안전 미구현	<code>dispatch.rs</code>	raw 바이트 직주입 — 타깃이 bracketed paste 모드면 200~/201~ 래핑하는 기존 붙여넣기 경로와 불일치(설계 문서 §4 약속 사항). WezTerm <code>send-text</code> 의미론 채택

2.2 기능 갭 (참조 설계 대비)

#	갭	참조
G7	<code>PaneInfo</code> 가 id/제목/크기뿐 — 종류(local/ssh/browser/sftp)·cwd·마지막 exit code·실행 중 명령· 활동 상태 없음	Herdr·iTerm2
G8	CLI 출력이 사람용 표뿐 — 에이전트가 표를 파싱해야 함	WezTerm <code>--format json</code>
G9	주소지정이 숫자 ID뿐 — 제목/cwd 매칭 불가	Kitty <code>--match</code>
G10	스폰 위치 지정 불가(항상 탭) — 분할/새 OS 창 선택 없음. SSH 스폰도 없음	WezTerm <code>split-pane</code>
G11	권한이 요청자 pane 전부-아니면-전무 — verb 단위 구분 없음, 승인 취소 UI 없음	Kitty rc-permission
G12	<code>focus / set-title</code> 없음 — 에이전트가 사용자 주의를 유도할 수단 없음	WezTerm <code>activate-pane</code>
G13	<code>notify</code> 없음 — 장기 작업 완료를 사용자에게 알릴 표준 경로 없음 (OSC 9는 자기 pane 한정)	cmux
G14	OSC 7771 in-band 채널 미구현 — 원격(SSH) pane에서 제어 발화 불가 (설계상 의도된 연기)	설계 문서 §2.2

#	갭	참조
G15	MCP 서버 미노출 — 에이전트가 매번 셸 명령으로 우회(토큰·왕복 비용, 파싱 부담)	CMUX Agent

3. 업그레이드 설계

3.1 [A] pane 메타데이터·활동 상태 (G7) — Herdr 모델

오케스트레이터에 `pane_meta.rs` 신설: pane별 최신 cwd(OSC 7)·실행 중 명령·마지막 CommandBlock (exit code·소요시간)·종류를 캐시(이미 흐르는 이벤트의 구독 집계 — 신규 신호 없음).

활동 상태 머신 (list 응답 + 탭 UI 배치 공용):

```
Idle      - 마지막 출력 후 2초 경과, 명령 경계 닫힘(OSC 133 D 수신)
Working   - 명령 실행 중(133 C 후 D 미수신) 또는 출력 진행 중
Blocked   - Working인데 N초(기본 15) 출력 정지 → 입력 대기 추정(프롬프트/암호/페이지)
Exited    - 프로세스 종료(exit code 보존)
```

PaneInfo 확장: `kind` (local/ssh/browser/sftp), `cwd`, `state`, `last_exit`, `running_cmd`, `title`, `cols×rows`. Blocked 판정은 휴리스틱이므로 응답에 `state_since_ms` 동봉(소비자가 재판정 가능).

3.2 [B] 머신 가독 출력·주소지정 (G8·G9)

- 모든 CLI verb에 `--json`: 응답 ControlResponse를 그대로 직렬화(이미 serde — 클라이언트 출력부만 분기).
- `--match "<k>:<v>[,..]"` 주소지정: `title`: (부분일치)·`cwd`: (접두)·`kind`: ·`state`: . 서버 측 해석(ListPanes 필터와 동일 코드), 매칭 0건/2건 이상이면 명시적 오류(모호성 거부 — Kitty 방식). 예: `nabi cli send --match "cwd:C:\proj,state:idle" --data "git pull r"`.

3.3 [C] 스폰 위치·SSH 스폰 (G1·G10)

- SpawnTerminal 에 `dock: tab|split-right|split-down|new-window` 추가(기본 tab). 분할/새 창은 AppCt1 로 도킹 적용(앱이 PaneSpawned 수신 시 위치 결정 — 기존 분할 경로 재사용).
- G1 수정과 결합**: `Command::SpawnLocalPane` 에 `reply: Option<u64>` (요청 seq) 추가, `Event::PaneSpawned` 에 seq 에코 → 제어 서버가 자기 seq의 PaneSpawned를 EventHub에

서 기다림(폴링 제거).

- `spawn --ssh <저장 세션>`: 자격증명 볼트 경유(평문 금지 원칙 유지), `Command::ConnectSsh` 재사용.

3.4 [D] 권한 정책 v2 (G4-G11) — Kitty 모델

verb를 3그룹으로 재분류하고 그룹별 정책:

그룹	verbs	off	ask	on
read	list, capture, wait, tail	거부	항상 허용	허용
act	spawn, resize, focus, set-title, notify, open-browser	거부	pane별 1회 승인	허용
inject	send, kill, open-sftp	거부	pane별 1회 승인(별도 집합 — act 승인이 inject를 풀지 않음)	허용

- 승인 토스트에 **요청 verb·타겟 표시**("pane 3이 pane 5에 입력 주입을 요청") + 허용 범위 선택(이번만/이 pane 항상/모두).
- 설정 다이얼로그에 승인 현황 **취소(revoke)** UI. 승인은 세션 한정(영속 안 함 — 보수 기본).
- wait/tail은 read로 이동(G4 해소). `[control] read = on` 기본(읽기는 ask 모드에서도 무승인).

3.5 [E] 사용자 상호작용 verbs (G12-G13)

- `focus --pane N` — 해당 pane 탭 활성화(+분리 창이면 창 전면). `AppCtl::Focus`.
- `set-title --pane N --title S` — 탭 제목 변경(기존 우클릭 이름변경과 동일 경로).
- `notify --title S --body S` — 토스트+데스크톱 알림. 기존 OSC 9/777 알림 파이프라인 재사용, 발신자 pane 표기("pane 3에서: 빌드 완료").

3.6 [F] MCP 서버 (G15) — 에이전트 네이티브 통합

`nabi.exe mcp` 서브커맨드: **stdio MCP 서버**로 떠서 제어 파이프의 프록시가 된다. Claude Code 쪽 등록은 `claude mcp add nabiterm -- nabi.exe mcp` 한 줄.

- 도구 세트 = ControlRequest와 1:1(`nabi_list_panes`, `nabi_capture`, `nabi_spawn`, ...) — 어휘 재사용, 신규 로직 없음. 응답은 §3.2의 JSON 그대로.
- 이점: 에이전트가 셸 왕복·표 파싱 없이 구조화 호출; `wait`를 MCP의 장기 도구 호출로 자연 표현.

- 정책은 동일 파이프 경유라 **자동 상속**(MCP라고 우회 없음). rust-mcp 계열 크레딧 1개 편 필요 (구현 시 결정; 의존 무거우면 NDJSON 직구현 — MCP studio는 JSON-RPC 한 겹이라 소형).

3.7 [G] capture-tail 품질 (G2·G6 + tmux 패리티)

- `capture --start <-N|줄> --end <줄>` (스크롤백 범위, tmux `-S/-E`), `--escapes` (SGR 보존 — 색/굵기 복원용, tmux `-e`), 응답에 `alt_screen: bool` 동봉(vim 등 전체화면 앱 판별).
- `tail` 을 **줄 델타 스트림**으로: pane별 누적 출력 줄 카운터(스크롤백 absolute line index) 기준 신규 줄만 전송. 80ms 폴링 유지하되 damage 이벤트로 깨우기(EventHub 재사용).
- `send`: 타깃 모델의 bracketed paste 상태 조회 후 200~/201~ 래핑(`--raw`로 우회 가능 — 제어문자 의도 주입용). 기존 paste 주입 방지 경로와 코드 공유.

3.8 [H] OSC 7771 (G14) — 마지막

설계 문서 §2.2 그대로(fire-and-forget verb만, 기본 off, 로컬 pane 한정). 원격 에이전트 유스케이스가 실제로 생기기 전까지 최후순위 — MCP(§3.6)가 로컬 유스케이스를 대부분 흡수한다.

4. 마일스톤 — 전체 구현 완료(2026-06-11)

단계	범위	상태
CP-5 정합성	G1(스폰 seq 회신)·G2(tail 델타)·G3(wait 페이로드)·G4(read 재분류)·G5(파이프 DACL)·G6(send paste 안전)	✅ 전부 구현·테스트(seq 정합 roundtrip, tail 델타 중복0/유실0 3종, wait exit code 회신, DACL SID 단위테스트)
CP-6 에이전트 가시성	§3.1 pane_meta+상태머신 · §3.2 <code>--json / --match</code> · 탭 UI ✨ 배지	✅ list <code>--json</code> 에 kind/cwd/state/state_ms/last_exit 노출, match 모호성 거부 테스트, 상태 전이 단위테스트
CP-7 동작 확장·정책 v2	§3.3 dock/SSH 스폰 · §3.4 verb 그룹 (read/act/inject) 정책+승인 그룹 표기 +설정 revoke UI · §3.5 focus/set-title/notify	✅ DockNext(split/new-window)·spawn <code>--ssh</code> (볼트 경유), 그룹별 별도 승인+revoke 테스트
CP-8 통합	§3.6 MCP 서버(<code>nabi mcp</code>) · §3.7 <code>capture --start/--end/--escapes+alt_screen</code> · §3.8 OSC 7771(opt-in, CP-4에서 선구현)	✅ MCP 12도구(initialize/tools/list/call), SGR 덤프 단위테스트

비고: §3.1의 Exited 상태는 레지스트리가 종료 pane을 즉시 제거(좀비 방지)하므로 list에 표시되지 않는다 — 종료 코드는 `wait --until exit (JSON {code})`로 회수한다.

순서 원칙: **결합 수정(CP-5) → 에이전트가 보는 것(CP-6) → 에이전트가 하는 것(CP-7) → 접속 방식(CP-8)**. 각 단계는 독립 배포 가능; CP-5는 프로토콜 비파괴(응답 필드 추가뿐), CP-6의 PanelInfo 확장은 serde 기본값으로 하위호환.

5. 모듈 매핑 (라인 게이트 250/400)

위치	모듈	변경
nabi-orchestrator	<code>pane_meta.rs</code> (신규)	이벤트 집계 캐시 + 상태머신(§3.1)
nabi-proto	<code>command.rs</code>	SpawnLocalPane에 <code>reply_seq</code> , <code>event.rs</code> PaneSpawned에 seq 에코
nabi-proto	<code>appctl.rs</code>	Dock/Focus/SetTitle/Notify variant 추가
nabi-control	<code>protocol.rs</code>	PanelInfo 확장·Match 타입·신규 verb(serde 기본값으로 하위호환)
nabi-control	<code>matcher.rs</code> (신규)	<code>--match</code> 해석·모호성 거부(§3.2)
nabi-control	<code>policy.rs</code>	verb 그룹 3분류·집합 분리·revoke(§3.4)
nabi-control	<code>tail_delta.rs</code> (신규)	줄 카운터 기반 델타 스트림(§3.7)
nabi-control	<code>pipe_acl.rs</code> (신규)	현재 사용자 SID DACL 생성(G5, windows-rs 보안 API)
nabi-control	<code>mcp.rs</code> (신규)	stdio JSON-RPC ↔ 파이프 프록시(§3.6)
nabi-app	<code>cli.rs</code>	<code>--json</code> / <code>--match</code> / 신규 verb 파싱(파일 분할: <code>cliparse.rs</code>)
nabi-app	<code>ctlapply.rs</code>	Dock/Focus/SetTitle/Notify 적용
nabi-app	<code>settings</code>	verb 그룹 정책 UI·승인 현황·revoke
nabi-vt	<code>model</code>	absolute line index 노출(tail 델타·capture 범위 공용)

6. 비범위 (여전히 제외)

- tmux `-cc` 식 전체 제어 모드(터미널 상태 전체 스트림) — 비용 대비 수요 불명.
- 다중 nabiTerm 인스턴스 간 제어·인스턴스 발견 — 파이프는 PID 스코프 유지.
- 승인 영속화(재시작 후 기억) — 보안 검토 후 별도 결정.
- 플러그인 API(M4) 통합 — ControlRequest 어휘 재사용 가능하게만 유지.

7. 참고 자료

- WezTerm CLI: [cli 개요](#) · [spawn](#) · [split-pane](#)
- Kitty: [remote control](#) · [rc 프로토콜](#) · [launch](#)
- [Herdr — 에이전트 상태 인지 멀티플렉서](#) · [해설](#)
- [cmux — 에이전트 오케스트레이션 터미널\(MCP\)](#)
- [tmux in the Coding Agents Era](#) · [tmux as AI agent runtime](#)